

Messages > Model capabilities

Batch processing

Process large volumes of Messages requests asynchronously with the Message Batches API, cutting costs by 50% and increasing throughput.

 Copy page 

Batch processing is a powerful approach for handling large volumes of requests efficiently. Instead of processing requests one at a time with immediate responses, batch processing allows you to submit multiple requests together for asynchronous processing. This pattern is particularly useful when:

- You need to process large volumes of data
- Immediate responses are not required
- You want to optimize for cost efficiency
- You're running large-scale evaluations or analyses

The Message Batches API is Anthropic's first implementation of this pattern.

 For how zero data retention (ZDR) applies to this feature, see [API and data retention](#).

Message Batches API

The Message Batches API is a powerful, cost-effective way to asynchronously process large volumes of [Messages](#) requests. This approach is well-suited to tasks that do not require immediate responses, with most batches finishing in less than 1 hour while reducing costs by 50% and increasing throughput.

You can [explore the API reference directly](#), in addition to this guide.

How the Message Batches API works

When you send a request to the Message Batches API:

1. The system creates a new Message Batch with the provided Messages requests.
2. The batch is then processed asynchronously, with each request handled independently.
3. You can poll for the status of the batch and retrieve results when processing has ended for all requests.

- Content moderation: Analyze large volumes of user-generated content asynchronously.
- Data analysis: Generate insights or summaries for large datasets.
- Bulk content generation: Create large amounts of text for various purposes (for example, product descriptions, article summaries).

Batch limitations

- A Message Batch is limited to either 100,000 Message requests or 256 MB in size, whichever is reached first.
- The system processes each batch as fast as possible, with most batches completing within 1 hour. You can access batch results when all messages have completed or after 24 hours, whichever comes first. Batches expire if processing does not complete within 24 hours.
- Batch results are available for 29 days after creation. After that, you may still view the Batch, but its results will no longer be available for download.
- Batches are scoped to a [Workspace](#). You may view all batches (and their results) that were created within the Workspace your request runs in.
- Rate limits apply to both Batches API HTTP requests and the number of requests within a batch waiting to be processed. See [Message Batches API rate limits](#). Additionally, processing may be slowed down based on current demand and your request volume. In that case, you may see more requests expiring after 24 hours.
- Because of high throughput and concurrent processing, batches may go slightly over your Workspace's configured [spend limit](#).
- Each batched request must have `max_tokens` of at least `1` . `max_tokens: 0` ([cache pre-warming](#)) is not supported inside a batch, because an ephemeral cache entry written during batch processing would likely expire before the follow-up request runs.

Supported models

All [active models](#) support the Message Batches API.

What can be batched

Almost any request you can make to the Messages API can be included in a batch. This includes:

- Vision
- Tool use, including all [server tools](#) (web search, web fetch, code execution, MCP connectors, advisor, and tool search)
- System messages
- Multi-turn conversations

Because each request in the batch is processed independently, you can mix different types of requests within a single batch.

A small number of Messages API parameters are **not** supported in batch requests. Including any of these returns a validation error:

Parameter	Why
<code>stream: true</code>	Batch results come back as a single file, not a stream.
<code>speed</code> (Fast mode)	Fast mode tunes synchronous latency, which doesn't apply to asynchronous batch processing.
<code>store</code> / <code>previous_thread_event_id</code> (Threads)	Threads are stateful; batch requests are not.
<code>cache_hint</code> / <code>context_hint</code>	These routing hints apply to synchronous request scheduling only.
<code>max_tokens: 0</code>	See Batch limitations .
<code>research_preview_2026_02: "active"</code>	Research preview mode is not available on the batch path.

💡 Because batches can take longer than 5 minutes to process, consider using the [1-hour cache duration](#) with prompt caching for better cache hit rates when processing batches with shared context.

Pricing

The Batches API offers significant cost savings. All usage is charged at 50% of the standard API prices.

Model	Batch input	Batch output
Claude Fable 5	\$5 / MTok	\$25 / MTok
Claude Mythos 5 (limited availability)	\$5 / MTok	\$25 / MTok
Claude Opus 5	\$2.50 / MTok	\$12.50 / MTok
Claude Opus 4.8	\$2.50 / MTok	\$12.50 / MTok

Claude Opus 4.6	\$2.50 / MTok	\$12.50 / MTok
Claude Opus 4.5	\$2.50 / MTok	\$12.50 / MTok
Claude Opus 4.1 (retired, except on Bedrock and Google Cloud)	\$7.50 / MTok	\$37.50 / MTok
Claude Opus 4 (retired, except on Google Cloud)	\$7.50 / MTok	\$37.50 / MTok
Claude Sonnet 5	\$1 / MTok	\$5 / MTok
Claude Sonnet 4.6	\$1.50 / MTok	\$7.50 / MTok
Claude Sonnet 4.5	\$1.50 / MTok	\$7.50 / MTok
Claude Sonnet 4 (retired, except on Bedrock and Google Cloud)	\$1.50 / MTok	\$7.50 / MTok
Claude Haiku 4.5	\$0.50 / MTok	\$2.50 / MTok
Claude Haiku 3.5 (retired, except on Bedrock and Google Cloud)	\$0.40 / MTok	\$2 / MTok

How to use the Message Batches API

Prepare and create your batch

A Message Batch is composed of a list of requests to create a Message. The shape of an individual request comprises:

- A unique `custom_id` for identifying the Messages request. Must be 1 to 64 characters and contain only alphanumeric characters, hyphens, and underscores (matching `^[a-zA-Z0-9_-]{1,64}$`).
- A `params` object with the standard [Messages API](#) parameters

You can [create a batch](#) by passing this list into the `requests` parameter:

[cURL](#)
[CLI](#)
[Python](#)
[TypeScript](#)
[C#](#)
[Go](#)
[Java](#)
[PHP](#)
[Ruby](#)



```
from anthropic.types.messages.batch_create_params import Request

client = anthropic.Anthropic()

message_batch = client.messages.batches.create(
    requests=[
        Request(
            custom_id="my-first-request",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=1024,
                messages=[
                    {
                        "role": "user",
                        "content": "Hello, world",
                    }
                ],
            ),
        ),
        Request(
            custom_id="my-second-request",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=1024,
                messages=[
                    {
                        "role": "user",
                        "content": "Hi again, friend",
                    }
                ],
            ),
        ),
    ],
)

print(message_batch)
```

In this example, two separate requests are batched together for asynchronous processing. Each request has a unique `custom_id` and contains the standard parameters you'd use for a Messages API call.

validation of the `params` object for each message request is performed asynchronously, and validation errors are returned when processing of the entire batch has ended. You can ensure that you are building your input correctly by verifying your request shape with the [Messages API](#) first.

When a batch is first created, the response has a processing status of `in_progress`.

Output



```
{
  "id": "msgbatch_01HkcTjaV5uDC8jWR4ZsDV8d",
  "type": "message_batch",
  "processing_status": "in_progress",
  "request_counts": {
    "processing": 2,
    "succeeded": 0,
    "errored": 0,
    "canceled": 0,
    "expired": 0
  },
  "ended_at": null,
  "created_at": "2024-09-24T18:37:24.100435Z",
  "expires_at": "2024-09-25T18:37:24.100435Z",
  "cancel_initiated_at": null,
  "results_url": null
}
```

Tracking your batch

The Message Batch's `processing_status` field indicates the stage of processing the batch is in. It starts as `in_progress`, then updates to `ended` once all the requests in the batch have finished processing, and results are ready. You can monitor the state of your batch by visiting the [Console](#), or using the [retrieval endpoint](#).

Polling for Message Batch completion

To poll a Message Batch, you'll need its `id`, which is provided in the response when creating a batch or by listing batches. You can implement a polling loop that checks the batch status periodically until processing has ended:

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()

MESSAGE_BATCH_ID = "msgbatch_01HkcTjaV5uDC8jWR4ZsDV8d"

message_batch = None
while True:
    message_batch = client.messages.batches.retrieve(MESSAGE_BATCH_ID)
    if message_batch.processing_status == "ended":
        break

    print(f"Batch {MESSAGE_BATCH_ID} is still processing...")
    time.sleep(60)
print(message_batch)
```

Listing all Message Batches

You can list all Message Batches in your Workspace using the [list endpoint](#). The API supports pagination, automatically fetching additional pages as needed:

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()

# Automatically fetches more pages as needed.
for message_batch in client.messages.batches.list(limit=20):
    print(message_batch)
```

Retrieving batch results

Once batch processing has ended, each Messages request in the batch has a result. There are four result types:

Result type	Description
successful	Request was successful. Includes the message result.
errored	Request encountered an error and a message was not created. Possible errors include invalid requests and internal server errors. You will not be billed for these requests.

cancel ed	User canceled the batch before this request could be sent to the model. You will not be billed for these requests.
expire d	Batch reached its 24-hour expiration before this request could be sent to the model. You will not be billed for these requests.

The batch's `request_counts` shows an overview of your results, indicating how many requests reached each of these four states.

Results of the batch are available for download at the `results_url` property on the Message Batch, and if the organization permission allows, in the Console. Because of the potentially large size of the results, it's recommended to stream results back rather than download them all at once.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby

```
client = anthropic.Anthropic()

# Stream results file in memory-efficient chunks, processing one at a time
for result in client.messages.batches.results(
    "msgbatch_01HkcTjaV5uDC8jWR4ZsDV8d",
):
    match result.result.type:
        case "succeeded":
            print(f"Success! {result.custom_id}")
        case "errored":
            if result.result.error.error.type == "invalid_request_error":
                # Request body must be fixed before re-sending request
                print(f"Validation error {result.custom_id}")
            else:
                # Request can be retried directly
                print(f"Server error {result.custom_id}")
        case "expired":
            print(f"Request expired {result.custom_id}")
```

The results are in `.jsonl` format, where each line is a valid JSON object representing the result of a single request in the Message Batch. For each streamed result, you can do something different depending on its `custom_id` and result type. Here is an example set of results:

.jsonl file

```
":"claude-opus-5","content":[{"type":"text","text":"Hello again! It's nice to see you. How can I assist you today? Is there anything specific you'd like to chat about or any questions you have?"}], "stop_reason":"end_turn", "stop_sequence":null, "usage": {"input_tokens":11, "output_tokens":36}}}] {"custom_id":"my-first-request", "result":{"type":"succeeded", "message": {"id":"msg_01FqfsLoHwgeFbguDgpz48m7", "type":"message", "role":"assistant", "model":"claude-opus-5", "content":[{"type":"text","text":"Hello! How can I assist you today? Feel free to ask me any questions or let me know if there's anything you'd like to chat about."}], "stop_reason":"end_turn", "stop_sequence":null, "usage": {"input_tokens":10, "output_tokens":34}}}}}]
```

If your result has an error, its `result.error` will be set to the standard [error shape](#).

Batch results may not match input order

Batch results can be returned in any order, and may not match the ordering of requests when the batch was created. In the preceding example, the result for the second batch request is returned before the first. To correctly match results with their corresponding requests, always use the `custom_id` field.

Canceling a Message Batch

You can cancel a Message Batch that is currently processing using the [cancel endpoint](#). Immediately after cancellation, a batch's `processing_status` will be `canceling`. You can use the same polling technique described earlier to wait until cancellation is finalized. Canceled batches end up with a status of `ended` and may contain partial results for requests that were processed before cancellation.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()

MESSAGE_BATCH_ID = "msgbatch_01HkcTjaV5uDC8jWR4ZsDV8d"

message_batch = client.messages.batches.cancel(
    MESSAGE_BATCH_ID,
)
print(message_batch)
```

The response shows the batch in a `canceling` state:

```
{
  "id": "msgbatch_013Zva2CMHLNnXjNJJkqJ2EF",
  "type": "message_batch",
  "processing_status": "canceling",
  "request_counts": {
    "processing": 2,
    "succeeded": 0,
    "errored": 0,
    "canceled": 0,
    "expired": 0
  },
  "ended_at": null,
  "created_at": "2024-09-24T18:37:24.100435Z",
  "expires_at": "2024-09-25T18:37:24.100435Z",
  "cancel_initiated_at": "2024-09-24T18:39:03.114875Z",
  "results_url": null
}
```

Using prompt caching with Message Batches

The Message Batches API supports prompt caching, allowing you to potentially reduce costs and processing time for batch requests. The pricing discounts from prompt caching and Message Batches can stack, providing even greater cost savings when both features are used together. However, because batch requests are processed asynchronously and concurrently, cache hits are provided on a best-effort basis. Users typically experience cache hit rates ranging from 30% to 98%, depending on their traffic patterns.

To maximize the likelihood of cache hits in your batch requests:

1. Include identical `cache_control` blocks in every Message request within your batch.
2. Maintain a steady stream of requests to prevent cache entries from expiring after their 5-minute lifetime.
3. Structure your requests to share as much cached content as possible.

Example of implementing prompt caching in a batch:

cURL CLI **Python** TypeScript C# Go Java PHP Ruby




```
client = anthropic.Anthropic()

message_batch = client.messages.batches.create(
    requests=[
        Request(
            custom_id="my-first-request",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=1024,
                system=[
                    {
                        "type": "text",
                        "text": "You are an AI assistant tasked with analyzing
literary works. Your goal is to provide insightful commentary on themes,
characters, and writing style.\n",
                    },
                    {
                        "type": "text",
                        "text": "<the entire contents of Pride and Prejudice>",
                        "cache_control": {"type": "ephemeral"},
                    },
                ],
            messages=[
                {
                    "role": "user",
                    "content": "Analyze the major themes in Pride and
Prejudice.",
                }
            ],
        ),
        Request(
            custom_id="my-second-request",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=1024,
                system=[
                    {
                        "type": "text",
                        "text": "You are an AI assistant tasked with analyzing
literary works. Your goal is to provide insightful commentary on themes,
characters, and writing style.\n",
                    },
                    {
                        "type": "text",
                        "text": "<the entire contents of Pride and Prejudice>",
                        "cache_control": {"type": "ephemeral"},
                    },
                ],
            ],
        ),
    ],
)
```

```
        "content": "Write a summary of Pride and Prejudice.",  
      },  
    ],  
  ),  
,  
],  
)
```

In this example, both requests in the batch include identical system messages and the full text of *Pride and Prejudice* marked with `cache_control` to increase the likelihood of cache hits.

Server tools and the agentic loop

All [server tools](#) (web search, web fetch, code execution, MCP connectors, advisor, and tool search) work in batch requests. The batch worker runs the same server-side agentic loop as the synchronous Messages API.

Because there is no open connection to maintain, the batch loop runs **more iterations per turn** than a synchronous request before it returns `stop_reason: "pause_turn"`. If a batch result comes back with `pause_turn`, the turn did not finish; you can continue it by submitting the paused assistant content in a follow-up request (batch or synchronous) exactly as shown in the [pause_turn continuation pattern](#).

The batch worker additionally throttles `web_search` per organization so that highly concurrent batch processing does not exhaust your organization's web-search rate limit. The batch retries throttled requests automatically; you don't need to handle this yourself, but very large web-search batches might take longer to complete.

Extended output (beta)

The `output-300k-2026-03-24` beta header raises the `max_tokens` cap to 300,000 for batch requests using Claude Opus 5, Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 5, or Claude Sonnet 4.6. Include the header to generate outputs far longer than the standard 128k `max_tokens` limit in a single turn.

 Extended output is available on the Message Batches API only, not the synchronous Messages API. It is supported on the Claude API and Claude Platform on AWS, and is not currently available on Amazon Bedrock, Google Cloud, or Microsoft Foundry.

Use extended output for long-form generation such as book-length drafts and technical documentation, exhaustive structured data extraction, large code-generation scaffolds, and long reasoning chains.

cURL

CLI

Python

TypeScript

C#

Go

Java

PHP

Ruby



```
from anthropic.types.beta.message_create_params import
MessageCreateParamsNonStreaming
from anthropic.types.beta.messages.batch_create_params import Request

client = anthropic.Anthropic()

message_batch = client.beta.messages.batches.create(
    betas=["output-300k-2026-03-24"],
    requests=[
        Request(
            custom_id="long-form-request",
            params=MessageCreateParamsNonStreaming(
                model="claude-opus-5",
                max_tokens=300_000,
                messages=[
                    {
                        "role": "user",
                        "content": "Write a comprehensive technical guide to
building distributed systems, covering architecture patterns, consistency
models, fault tolerance, and operational best practices.",
                    }
                ],
            ),
        ],
    ),
)

print(message_batch)
```

Best practices for effective batching

To get the most out of the Batches API:

- Monitor batch processing status regularly and implement appropriate retry logic for failed requests.
- Use meaningful `custom_id` values to easily match results with requests, since order is not guaranteed.
- Consider breaking very large datasets into multiple batches for better manageability.
- Dry run a single request shape with the Messages API to avoid validation errors.

If experiencing unexpected behavior:

- Verify that the total batch request size doesn't exceed 256 MB. If the request size is too large, you may get a 413 `request_too_large` error.
- Check that you're using [supported models](#) for all requests in the batch.
- Ensure each request in the batch has a unique `custom_id`.
- Ensure that it has been less than 29 days since batch `created_at` (not processing `ended_at`) time. If over 29 days have passed, results will no longer be viewable.
- Confirm that the batch has not been canceled.

Note that the failure of one request in a batch does not affect the processing of other requests.

Batch storage and privacy

- **Workspace isolation:** Batches are isolated within the Workspace they are created in. They can only be accessed by API requests in that same Workspace, or users with permission to view Workspace batches in the Console.
- **Result availability:** Batch results are available for 29 days after the batch is created, allowing ample time for retrieval and processing.

Data retention

Batch processing stores request and response data for up to 29 days after batch creation. You can delete a message batch at any time after processing using the `DELETE /v1/messages/batches/{batch_id}` endpoint. To delete an in-progress batch, cancel it first. Asynchronous processing requires server-side storage of both inputs and outputs until batch completion and result retrieval.

For ZDR eligibility across all features, see [API and data retention](#).

How long does it take for a batch to process? >

Is the Batches API available for all models? >

Can I use the Message Batches API with other API features? >

How does the Message Batches API affect pricing? >

Can I update a batch after it's been submitted? >

Are there Message Batches API rate limits and do they interact with the Messages API rate limits? >

How do I handle errors in my batch requests? >

How does the Message Batches API handle privacy and data separation? >

Can I use prompt caching in the Message Batches API? >

Next steps



Search results

Enable natural citations for RAG applications by providing search results with source attribution.



Prompt caching

Reduce cost and latency by caching prompt prefixes shared across requests in a batch.

Cette page vous a-t-elle été utile ?



Claude Platform Docs



Solutions

AI agents

Code modernization

Coding

Customer support

Financial services

Government

Higher education

K-12 teachers

Life sciences

Partners

Claude on AWS

Claude on Google Cloud

Help and security

Availability

Status

Support

Discord

Learn

Blog

Courses

Use cases

Connectors

Customer stories

Engineering at Anthropic

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Careers

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

[Responsible disclosure policy](#)

[Terms of service: Commercial](#)

[Terms of service: Consumer](#)

[Usage policy](#)